

so my freind had an interview tomorrow, and i tolf him to play the man,more than the game, bec it is qualcomm and his interviews are

<https://www.linkedin.com/in/kashifinayat/> <https://www.linkedin.com/in/saad-waheed/> <https://www.linkedin.com/in/xavi-salv%C3%A0-grimalt-a7b783214/> https://careers.qualcomm.com/careers/job/446718785409?hl=en-US&domain=qualcomm.com&source=APPLICANT_SOURCE-6-2 Relevant Projects:

<https://github.com/124107157-KV/TinySoC> https://github.com/124107157-KV/RV32I_Processor https://github.com/124107157-KV/SERV/tree/serv_nn_layer <https://github.com/124107157-KV/AES-128-Encryption-Core-with-UVM-Verification>

I have an interview tomorrow at Qualcomm I have provided the LinkedIn profiles of the interviewers. I have provided the exact job URL of the role to which I will be interviewing for. The interview is for an hour. I have also provided my resume for reference. I have provided all the relevant projects which I worked on which meets the requirements related to the job description. I would like to play the game, but I would also like to play the man and impress them and get the job. and his chatgpt gave him tge following response, i want to understand it in kinda easier rds to explait t him I'll turn this into a practical one-hour interview game plan: role decoding, interviewer-specific angles, your strongest project stories, and the exact narrative to steer the conversation toward "hire him." I found the role text through public job mirrors because the Qualcomm careers page is mostly dynamic. The core signal: this is CPU DV, not generic ASIC DV — they will reward architecture + verification methodology + debug discipline. Thought for 1m 9s Your best strategy tomorrow: don't present yourself as "a junior trying to enter CPU DV." Present yourself as "an engineer who has already touched CPU RTL, RISC-V ISA behavior, verification environments, regressions, GLS, automation, and debug — now wants to scale that into Qualcomm-class CPU verification."

That is the game. The man-to-man part is: each interviewer should feel you understand their world. 1. What Qualcomm is actually interviewing you for This role is not just "SystemVerilog/UVM." It is CPU verification with architecture awareness. The job asks for someone who can work with CPU/SOC architects, develop test and coverage plans, build scalable verification methodology, develop stimulus/checkers/assertions/trackers/coverage, run regressions, debug failures, and track DV progress using coverage and bug metrics. It also explicitly values knowledge of cache coherence, memory ordering, prefetching, branch prediction, renaming, speculative execution, MMU/address translation, RIS/random instruction sequencing, assembly, simulators, coverage, GLS, waveform debug, and formal tools. So your message should be: "I'm strongest at the intersection of CPU microarchitecture, verification, automation, and debug. I have not only

studied RISC-V; I modified a RISC-V core, verified execution-unit behavior, built UVM infrastructure, automated regressions, and I'm actively extending into GLS and performance instrumentation." That lines up very well with your CV: RISC-

V/SERV execution-unit modification, FPGA validation, Yosys GLS, UVM AES environment with 561 transactions and 99.82% coverage, Python automation, C/C++, Verilog/SystemVerilog, and regression/debug experience. 2. Interviewer strategy: "play the man" cleanly Do not fake knowing their confidential work.

Instead, show that you studied their public profile and can speak their engineering language. Kashif Inayat Public snippets show Kashif as a Senior Engineer in Qualcomm's Custom CPU Team, with PhD background and interests around CPU microarchitecture, computer arithmetic, ML accelerators/TPUs, and power. He has also publicly posted about Qualcomm Cork/Oryon CPU hiring and CPU design, verification, performance, and power. How to impress him: connect

your SERV project to microarchitecture trade-offs, not just "I added stochastic logic." Say something like: "What interested me in SERV was the trade-off space:

tiny bit-serial datapath, area and power constraints, and how to extend functionality without breaking RV32I behavior. I tried to preserve baseline ISA transparency while adding runtime-selectable paths. That made me think more carefully about control, writeback, verification boundaries, and performance versus precision trade-offs." That is far better than saying: "I added AI to RISC-V."

That sounds fluffy. The above sounds like an engineer. Saad Waheed Public snippets show Saad as a Staff Engineer at Qualcomm and an experienced Design

Verification Engineer. Other public profile information associated with Saad Waheed emphasizes verification engineering, test plans, RTL debug, simulation improvements, and SiFive/core coverage closure experience. How to impress him: talk like a DV engineer: test plan, assertions, scoreboard, coverage closure, regressions, debug triage. Say: "For my AES UVM project, I treated it like a real

verification problem: define directed and constrained-random sequences, monitor DUT behavior, compare against a bit-exact reference model, use coverage to guide closure, and debug mismatches through waveform and transaction-level logs." Your AES repo publicly reports 561 passing transactions,

zero UVM errors/fatals, and 99.82% total functional coverage. Xavi Salvà Grimalt Public snippets show Xavi as a Computer Architect at Qualcomm. How to impress

him: speak in architecture language: correctness, observability, performance counters, corner cases, memory behavior, instruction classes. Say: "One area I'm actively building is connecting functional correctness with performance visibility

— for example, adding cycle-counting and instruction-class instrumentation to simulation logs so that verification is not only pass/fail but also gives insight into behavior under different workloads." That links directly to the role's

“performance benchmarks/workloads” preference and your CV’s performance instrumentation angle. 3. Your strongest project order tomorrow Do not start with TinySoC. Start with CPU DV relevance. Project 1: SERV RISC-V Core — your main weapon Use this as your anchor. Your repo says the SERV fork extends the bit-serial RISC-V core with a dual-mode ALU, runtime stochastic/MAC controls, NN-friendly serial MAC, and preserves baseline RV32I behavior when SC/NN modes are disabled. Your pitch: “My most relevant project is my SERV RISC-V core extension. I worked on a bit-serial RV32I core, modified the execution path, added a dual-mode ALU, and focused on preserving baseline RV32I behavior when the new mode was disabled. From a verification perspective, the key challenge was not just adding logic — it was proving that normal instruction behavior remained correct, checking writeback/control behavior, and validating the modified datapath through simulation and FPGA.” If they push you: talk about ALU ops, control signals, writeback muxing, comparator behavior, deterministic vs stochastic mode, and what you would verify first. Project 2: AES-128 UVM — your DV credibility Your repo shows a complete UVM flow with agent, driver, monitor, scoreboard, bit-exact reference model, constrained-random and directed testing, coverage, waveform debug, and 99.82% coverage. Pitch: “The AES UVM project is where I demonstrate verification methodology: driver, monitor, scoreboard, reference model, constrained-random plus directed tests, coverage-driven closure, and waveform debug. It taught me how to think beyond ‘does the RTL run?’ and toward reusable verification infrastructure.” Project 3: TinySoC — your end-to-end silicon flow differentiator TinySoC shows RTL-to-GDSII using LibreLane/Yosys/OpenROAD/Magic/Netgen/KLayout, with UART/timer, RAM, ROM, bus interconnect, timing/utilization analysis, and 50 MHz target. Pitch: “TinySoC is not my main CPU DV project, but it shows I understand the broader RTL-to-implementation path. I built a small CPU-style SoC with UART, timer, RAM, ROM, bus interconnect, and took it through open-source synthesis, PnR, DRC/LVS, and GDS inspection. That helps me understand why clean RTL, timing, reset, clocks, and verification assumptions matter downstream.” Project 4: RV32I Processor — backup for ISA fundamentals Your RV32I repo describes a simplified 32-bit RISC-V processor supporting arithmetic, logic, memory access, and control-transfer instructions. Use it only if they ask basic RISC-V/processor implementation questions. 4. Your 60-second opening answer Use this when they say: “Tell me about yourself.” “I’m an embedded and verification engineer based in Cork, currently working in ASIC validation, and my main direction is CPU design verification. My strongest match for this role is the combination of RISC-V microarchitecture work, RTL verification, UVM, C/C++, Python automation, and debug. In my master’s thesis, I modified the SERV RV32I core execution path,

added a dual-mode ALU/stochastic extension, and focused on validating datapath correctness while preserving baseline ISA behavior. Separately, I built a complete UVM environment for an AES-128 core with driver, monitor, scoreboard, reference model, assertions, constrained-random testing, and coverage closure, reaching 99.82% functional coverage with zero errors. In my current ASIC validation role, I build Python/C/C++ automation for regressions, protocol-level debug, and timing/signal analysis. So I'm not coming only from academic RTL — I'm used to debugging real hardware/software interaction failures and building repeatable validation flows. What I'm looking for now is to apply that foundation to high-performance CPU DV at Qualcomm." Clean. Confident. No desperation. Tiny brag, but properly clock-gated. 5. Questions they may ask — and how you should angle the answer "What did you actually verify in SERV?" Answer in layers: "First, baseline deterministic behavior: normal RV32I path should remain unchanged when stochastic/MAC modes are disabled. Second, control correctness: mode selection, ALU result selection, writeback path, comparator path. Third, datapath correctness: ADD/SUB/logic/compare behavior versus expected outputs. Fourth, integration behavior: ensuring added muxing or mode controls do not corrupt the original SERV flow. If I were extending it further for industrial DV, I would add instruction-stream testing, assertions around illegal mode combinations, scoreboard comparison against an ISA model, and coverage over instruction classes and mode transitions." "How would you verify a CPU block?" Say: "I would start from the microarchitecture spec, define features and failure modes, then build a test plan with functional coverage points. For a block-level unit, I'd define legal/illegal stimulus, directed corner cases, constrained-random testing, assertions for protocol/control invariants, checkers or scoreboard for architectural correctness, and regression metrics. For CPU-level verification, I would add instruction-stream generation, reference-model comparison, exception/interrupt behavior, memory-ordering scenarios, and coverage closure across instruction classes and microarchitectural states." "What is coverage closure?" Say: "Coverage closure is not just hitting a percentage. It means understanding what behavior has been observed, what remains untested, whether the missing bins are meaningful or unreachable, and whether directed or constrained-random stimulus should be improved. I prefer using coverage as a debug tool, not a decoration." Nice line. Use it. "Why Qualcomm?" Say: "Qualcomm is one of the few places where CPU design, verification, power, performance, and real product impact meet at scale. The Cork CPU team is especially interesting to me because the work is not isolated block verification — it connects architecture, microarchitecture, workloads, regressions, and real silicon readiness. My background is smaller-scale, but it maps directly: RISC-V

RTL modification, UVM methodology, Python regressions, C/C++ firmware, GLS, and hardware debug. I want to grow in a team where correctness, performance, and engineering discipline all matter.”

6. High-value technical topics to revise tonight Focus on these. Do not scatter into every topic under the sun. CPU DV core topics: test plan structure, functional coverage, assertions, scoreboard, monitor/driver, constrained-random testing, RIS/random instruction sequences, reference model, regression debug, waveform debug, GLS basics. CPU architecture topics: pipeline stages, fetch/decode/execute/writeback, branch prediction, renaming, ROB/retire, load-store queue, cache hierarchy, MMU/TLB, memory ordering, exceptions/interrupts, speculation recovery. RISC-V topics: RV32I instruction formats, load/store, branches/jumps, ALU ops, register x0, immediate encoding, misaligned access, traps/exceptions, simple 5-stage pipeline hazards. Power/performance topics: IPC, CPI, stalls, branch mispredict penalty, cache miss penalty, clock gating, power gating, DVFS/DCVS, throttling, workload counters.

7. Smart questions to ask them Ask 2–3 only. Make them feel you are already thinking like a teammate. “For this role, is the team more focused on block-level CPU DV, subsystem-level verification, or performance/workload-driven verification?” “How does the team usually measure DV progress — mainly functional coverage, bug burn-down, regression pass rate, architectural test milestones, or a mix?” “For someone joining with RISC-V/RTL/UVM and validation automation experience, what CPU DV area would you expect them to ramp into first?” For Kashif/Xavi specifically: “How closely does the DV team interact with architects when converting microarchitectural intent into checkers, coverage, and tests?” That question is gold. It shows maturity.

8. What not to do Do not oversell SERV as comparable to Qualcomm Oryon. Say it is a learning and implementation project on a small RISC-V core. Do not say “I know CPU architecture deeply” unless you can defend renaming, memory ordering, speculation, TLBs, and cache coherence. Say: “I have hands-on fundamentals and I’m actively building depth.” Do not spend too much time on stochastic computing unless they ask. For this role, the relevant part is: you modified execution logic and verified correctness. Do not talk like a firmware-only candidate. Your embedded background is useful only when framed as debug, automation, C/C++, regressions, protocol-level thinking, and hardware/software boundary experience.

9. Your final positioning statement Carry this in your head: “I am not just a UVM beginner and not just an embedded engineer. I am a CPU-DV-transition candidate with hands-on RISC-V RTL modification, UVM verification, Python regression automation, C/C++ debug, and silicon-validation discipline. I can ramp fast because I understand both the design side and the verification/debug side.” That is the storyline. Tomorrow, don’t try to sound like

you already worked on a Qualcomm CPU. Sound like someone they can trust near one.

I actually think this advice is pretty good—but it's written like a consultant's report. 😊 The core idea is much simpler.

If I had to explain it to your friend over coffee, I'd say this:

What does "play the man, not just the game" actually mean?

Most people prepare like this:

"Let me study UVM."

"Let me study RISC-V."

"Let me memorize branch prediction."

That's **playing the game**.

But Qualcomm interviews, especially senior engineers, are also about:

"Would I enjoy working with this person every day?"

"Does this person think like one of us?"

That's **playing the man**.

You don't manipulate them.

You simply speak in a language that makes each interviewer think,

"This guy understands the problems I deal with."

Qualcomm is NOT hiring someone who knows SystemVerilog.

They're hiring someone who can think.

Imagine they build Ferrari engines.

They don't want someone who says

"I know how to use a wrench."

They want someone who says

"If the engine overheats, here's how I'd figure out why."

That's the difference.

What each interviewer probably cares about

Interviewer 1 (Kashif)

He is close to CPU architecture.

He thinks about

- processors
- performance
- power
- architecture decisions

So don't only say

"I modified SERV."

Instead say

"While modifying SERV I started appreciating the trade-offs between keeping the processor small while adding new functionality."

That tells him

"This person thinks like an architect."

Interviewer 2 (Saad)

He's verification.

His daily life is basically

Bugs.

Bugs.

More bugs.

He doesn't care that you wrote code.

He cares that you know

- how you found bugs
- how you proved bugs were fixed
- how you verified correctness

Instead of saying

"I made an AES UVM project."

Say

"I built a driver, monitor, scoreboard, ran constrained random tests, checked coverage, and debugged mismatches until the design behaved correctly."

Now he hears

"This person already thinks like a DV engineer."

Interviewer 3 (Xavi)

He's closer to architecture again.

He'll likely enjoy discussions like

Why is this instruction behaving this way?

Why does performance drop?

What corner cases exist?

He'll appreciate curiosity.

Why they said not to oversell SERV

Imagine someone built a toy Formula 1 car.

Then in Ferrari's interview they say

"I basically built an F1 engine."

Ferrari knows that's nonsense.

Instead say

"It was a smaller project, but it taught me how changing one execution unit affects the entire processor."

That's believable.

Engineers LOVE believable.

Why they want him to start with SERV

His projects are roughly:

SERV ★★★★★

AES UVM ★★★★★

TinySoC ★★★

RV32I ★★

The Qualcomm role is CPU Design Verification.

Which project screams CPU?

SERV.

So start there.

What they mean by "story"

Don't list projects like a resume.

Tell a journey.

Instead of

I did SERV.

Then AES.

Then TinySoC.

Say

"I became interested in processor verification while modifying the SERV RISC-V core. That taught me how CPU logic changes affect correctness. I then wanted to strengthen my verification methodology, so I built an AES UVM environment. Finally I wanted to understand how designs move beyond RTL, so I built TinySoC."

Now every project leads naturally to the next.

It feels intentional.

Why they keep saying "debug"

Because engineers spend WAY more time debugging than writing RTL.

Anyone can write code.

The valuable engineer can answer

Why did this fail?

Where exactly is the failure?

How would you isolate it?

Every project should include

"I faced this problem..."

"I investigated..."

"I found..."

"I fixed..."

That's real engineering.

Why they keep mentioning coverage

Coverage is basically answering

"How sure are you that your processor actually works?"

Getting

99%

doesn't automatically mean

everything works.

Good engineers ask

What's missing?

Why is it missing?

Is it an unreachable case?

That's called thinking.

The "Tell me about yourself" answer

The suggested answer is good because it follows a simple formula:

Past

"I've worked on CPU architecture and verification."

↓

Present

"I currently do ASIC validation and automation."

↓

Future

"I now want to move into Qualcomm CPU Design Verification."

Very clean.

The smartest advice in that whole response

This line:

Don't sound like you've already worked on Qualcomm CPUs. Sound like someone they can trust near one.

That is excellent advice.

Interviewers don't expect graduates or early-career engineers to know everything.

They *do* expect honesty.

If your friend says

"I don't know that yet, but here's how I'd approach learning it..."

that's often stronger than pretending to know.

The biggest takeaway I'd give your friend

Forget trying to impress them with buzzwords.

Every answer should quietly communicate four things:

1. **I understand how processors work.**
2. **I know how to verify and debug designs systematically.**
3. **I enjoy solving engineering problems, not just writing code.**
4. **I'm honest about what I know, and I learn quickly.**

If each interviewer finishes the hour thinking, *"I'd trust this person to investigate a failing regression and work through it methodically,"* he's done far better than someone who simply recites textbook definitions.

That's the real meaning behind "playing the man, not just the game."

just so you know I don't have actual work experience related to the job role and the projects I did are partly hobby projects and SERV is my masters thesis. None of these are my recent projects, its been a year since I worked on them and know about them.

That context actually changes my advice a bit, and I think it's important.

The biggest mistake your friend could make tomorrow is trying to **sound like a Qualcomm engineer already**. Experienced interviewers can usually tell when someone is stretching their experience within a few minutes.

Instead, he should lean into what he *does* have.

Here's how I would position it

Instead of thinking:

"I need to convince them I've already done CPU verification."

Think:

"I need to convince them I have a solid foundation and can become productive quickly."

Those are two very different goals.

What he should openly acknowledge

If they ask,

"Tell us about your experience."

He shouldn't say:

"I've worked extensively on CPU verification."

Because he hasn't.

Instead:

"Most of my CPU verification experience comes from my master's thesis on the SERV RISC-V core and a few personal projects. Professionally, I haven't yet worked in a dedicated CPU Design Verification role, which is exactly why I'm excited about this opportunity."

Notice what happened?

He admitted the gap...

...without apologizing for it.

About SERV

Since it's been a year, they are **not** expecting him to remember every signal name or every line of RTL.

They *are* expecting him to remember:

- Why he chose the project
- What problem he solved
- How he approached it
- The biggest challenge
- What he learned

If they ask

"How did your ALU modification work?"

and he genuinely can't remember every implementation detail,

a perfectly respectable answer is:

"It's been about a year since my thesis, so I don't want to guess the exact implementation. What I do clearly remember is the design objective, the verification approach, and the trade-offs we were making."

That's far better than inventing details.

Hobby projects are not a weakness

Lots of Qualcomm candidates—especially for graduate or early-career roles—bring university and personal projects.

The difference is **how they present them**.

Don't say:

"It was just a hobby project."

Say:

"This was a self-driven project that I used to learn UVM methodology."

or

"This project helped me understand RTL verification and debugging beyond coursework."

"Just a hobby" unintentionally downplays the work.

What worries me most

You said:

"It's been a year since I worked on them."

That is honestly the biggest risk.

Not the lack of experience.

If I were interviewing him, I'd probably ask things like:

- Why did you implement it that way?
- What was the hardest bug?
- Why did your scoreboard work like that?
- Why was this assertion needed?

If he can't answer those, it will be noticeable.

So tonight, I'd spend **much more time revisiting the projects** than memorizing theory.

For each project, he should be able to answer these six questions:

1. **What was the objective?**
2. **What exactly did you contribute?**
3. **What was the hardest challenge?**
4. **How did you debug it?**
5. **If you started again today, what would you improve?**
6. **What did you learn?**

Those six questions cover a huge portion of project discussions.

One thing I'd change from the original ChatGPT advice

The original advice occasionally makes him sound a little too experienced.

For example, this:

"I'm strongest at the intersection of CPU microarchitecture, verification, automation, and debug."

I probably wouldn't say that.

I'd soften it to something like:

"My strongest foundation is in RISC-V architecture, RTL design, verification methodology through academic and personal projects, and automation. I'm now looking to build on that foundation in an industrial CPU Design Verification environment."

That sounds more authentic.

My final advice

If I had to give him one sentence to remember before walking into the interview, it would be this:

Don't try to convince them you're already a Qualcomm CPU verification engineer. Convince them you're the kind of engineer

they'd enjoy teaching, because you've already shown curiosity, discipline, and the ability to learn through challenging projects.

That mindset is much easier to sustain in an interview, and it comes across as genuine rather than rehearsed.